

第三章 词法分析（Lexical Analysis）

复习速查表 · 覆盖第一讲与第二讲全部知识点

3.1 词法分析器（Scanner）的功能

基本功能（第一讲 p.3）

- 输入源程序字符串，输出**单词符号（Token）**序列
- 删去空格字符和注释等无用符号
- 关键：找出符号的**分割符**

单词的分类（第一讲 p.4-5）

类别	说明
关键字（Keyword）	语言自身使用的单词，通常不允许作标识符，又称 保留字（Reserved Word）
标识符（Identifier）	表示变量名、函数名等的合法串
常数（Constant）	整型、实型、布尔型、字符型等
运算符（Operator）	算术运算符、逻辑运算符、关系运算符等
界限符（Delimiter）	，；（）:= 等，用于分隔或运算

单词的机内表示——二元式（第一讲 p.6）

二元组格式

(单词种别码, 单词自身值)

种别码规则：

- 关键字：一种一码
- 标识符：统一一种码
- 常数：一类型一码
- 界限符：一符一码

属性值规则：

- **无自身值**：关键字、界限符、运算符（种别码本身唯一确定）
- **有自身值**：常数（数值本身）、标识符（符号表入口地址）

经典示例（第一讲 p.7）：while(*pointer!='\0'){pointer++;}

```
while      → (WHILE, _)
(          → (SLP, _)
*          → (FETCH, _)
pointer    → (IDN, 符号表入口指针)
!=         → (RELOP, NE)
'\0'       → (CONST, 0)
)          → (SRP, _)
{          → (LP, _)
pointer    → (IDN, 符号表入口指针)
++         → (INC, _)
;          → (SEMI, _)
}          → (RP, _)
```

相关问题（第一讲 p.8, p.10）

- 超前扫描（Lookahead）：识别双字符运算符（**、/*、:=）时需多读一个字符，如 FORTRAN 中 DO 90 k=1,10（循环）vs. DO 90 k=1.10（赋值）
- 预处理：剔除空格、换行、注释
- 词法错误处理：恐慌模式（panic mode）——删字符、插字符、字符替换

扫描器运行方式（第一讲 p.9）

- 作为独立的遍：简单，但需额外开销
- 作为独立子程序：由语法分析器调用 getNextToken()，节省内存

符号表（Symbol Table）

作用（第一讲 p.11-12）

阶段	主要用途
词法分析	建立符号表，填入标识符/常数的名字、类型、数值，写入 TOKEN 字
语法分析	查找符号表（读取符号信息）
语义分析/中间代码	查填符号表，填入类型信息，用入口地址代替变量名
数据存储分配	将数据区映像地址写入 ADDR 栏
代码优化	查找 VAL 栏检查变量是否为常数
目标代码生成	查找符号表，暂存引用活跃信息

内容字段（第一讲 p.13）

NAME（字符串表起始位置+长度）、TYPE（整/实/布尔/字符）、KIND（变量/常数名/数组名/过程名）

数据结构示意（第一讲 p.14）

符号表 symtable 由两部分组成：

- lexptr：指向字符串数组 lexemes 中标识符起始位置
- （Token 种别）及 属性字段

3.2 词法单元的识别——正则表达式与状态转换图

正则表达式 (Regular Expression, RE) (第一讲 p.17)

设 Σ 为字母表, RE 递归定义如下:

RE 定义

- 1. Φ 是 RE, $L(\Phi) = \Phi$
- 2. ε 是 RE, $L(\varepsilon) = \{\varepsilon\}$
- 3. $\forall a \in \Sigma, a$ 是 RE, $L(a) = \{a\}$
- 4. 若 r, s 是 RE, $L(r) = R, L(s) = S$, 则:
 - $(r|s)$ 是 RE, $L(r|s) = R \cup S$ (并)
 - (rs) 是 RE, $L(rs) = RS$ (连接)
 - (r^*) 是 RE, $L(r^*) = R^*$ (Kleene 闭包)

运算优先级 (第一讲 p.20)

$* > \text{连接} > |$

- $|$ 具有交换律、结合律
- 连接具有结合律及对 $|$ 的分配律
- 圆括号指定优先关系

扩展运算符 (第一讲 p.19)

符号	等价形式 / 含义
r^+	rr^* (一个或多个)
$r?$	εr (零个或一个)
$[a_1a_2 \dots a_n]$	$a_1 a_2 \dots a_n$ (字符类)
$[a-e]$	$a b c d e$ (字符范围)

注: 扩展运算符只增加简洁性, 不扩展 RE 的描述能力。

正则定义示例 (第一讲 p.18, p.21)

C 语言标识符

```
letter_ -> A|B|...|Z|a|b|...|z|_  
digit   -> 0|1|...|9  
id       -> letter_ ( letter_ | digit )*
```

数字常量 (含浮点)

```
digit    -> [0-9]  
digits   -> digit+  
number   -> digits ( . digits )? ( E [+-]? digits )?
```

关系运算符词法单元模式（第一讲 p.21-22）

`relop -> < | > | <= | >= | = | <>`

属性值：LT / GT / LE / GE / EQ / NE

状态转换图（Transition Diagram）（第一讲 p.23-26）

- **状态（State）**：表示识别词素过程中可能出现的情况，看作”已处理部分的总结”
- **接受状态/终态**：表示已找到词素；带 * 表示最后读入的符号**不在**词素中（需回退）
- **开始状态**：用 start 边标注
- **边（Edge）**：标号为一个或多个输入符号，从当前状态按符号跳转

relop 状态转换图简述（第一讲 p.25）

- 状态 0 (start)：读到 < → 1，读到 = → 5（返回 EQ），读到 > → 6
- 状态 1：读到 = → 2（返回 LE），读到 > → 3（返回 NE），其他 → 4*（返回 LT，回退）
- 状态 6：读到 = → 7（返回 GE），其他 → 8*（返回 GT，回退）

保留字与标识符的区分（第一讲 p.26）

1. 在符号表中**预填保留字**，并标明不是普通标识符
2. 为每个关键字建立**单独状态转换图**，设定优先级高于标识符

3.3 词法分析器的设计与实现

从转换图构造词法分析器（第二讲 p.3）

- 变量 `state` 记录当前状态
- 用 `switch(state)` 跳转至各状态对应代码段
- 每段代码读入字符 → 确定下一状态；找不到转移边则调 `fail()` 错误恢复
- 进入接受状态：返回对应词法单元；若状态有 * 标记，先**回退 forward 指针**

处理多个模式的方法（第二讲 p.6）

1. **顺序尝试**：按优先级逐一启动各状态图，失败则回退并尝试下一个
2. **并行运行**：同时运行多个状态图，用 `greedy`（最长匹配）策略选取最长匹配前缀
3. **合并为一图**：预先将所有状态图合并成单一状态转换图一次运行（效率最高）

关键数据结构与子例程（第二讲 p.8）

名称	作用
ch	当前输入字符
token	输入缓冲区（字符数组），累积当前单词字符
symbol	单词种别（子程序返回值）
attr	属性（全局变量）
Lookup(token)	将 token 存入符号表，返回入口指针
isKeyword(token)	判别是否关键字，返回关键字种别或 -1
getchar()	从输入缓冲区读入一个字符
isLetter()/isalpha()	判断字母
isdigit(ch)	判断数字

状态图实现算法概要（第二讲 p.9-11）

scan() 主流程伪代码

```
1. getchar()
2.   : WHILE ch == ' ' DO getchar()
3. CASE ch OF
    isdigit(ch):
        token;  getchar()
        ;      -> attr; return NUM
    isalpha(ch):
        /   token

        key = isKeyword(token)
        IF key>=0 THEN return key      //
        Lookup(token) -> attr; return IDN
    ':' : getchar(); IF ch=='=' THEN return ASG;
    '+' -> ADD,  '-' -> SUB,  '*' -> MUL,  '/' -> DIV
    '=' -> EQ,   '>' -> GT,   '<' -> LT
    '(' -> LP,   ')' -> RP,   ';' -> SEMI
    ->
```

relop 对应代码结构（第二讲 p.5）

getRelop() 代码骨架

```
TOKEN getRelop() {
    TOKEN retToken = new(RELOP);
    while(1) {
        switch(state) {
            case 0: c=nextChar();
                    if(c=='<') state=1;
                    else if(c=='=') state=5;
                    else if(c=='>') state=6;
                    else fail();
                    break;
            case 1: ... //  <=, <>, <
            ...
        }
    }
}
```

```

        case 8: retract();          //  *,
            retToken.attribute = GT;
            return(retToken);
        }
    }
}

```

LEX 简介

实现过程（第二讲 p.14）

Lex 源程序 $\xrightarrow{\text{Lex 编译器}}$ lex.yy.c $\xrightarrow{\text{C 编译器}}$ 词法分析器 L

输入流 $\xrightarrow{\text{词法分析器 L}}$ Token 序列

Lex 程序结构（第二讲 p.15-16）

```

%{
    (C )
%}

%%
    :    { }
%%

```

识别规则格式

每条规则形如 token_i {action_i}, 匹配时执行对应动作（如输出 Token）。

Lex 程序片段示例（第二讲 p.16-17）

```

digit  [0-9]
alpha  [a-zA-Z]
alnum  [a-zA-Z0-9]
%%
begin      {OUT(BEGIN,NULL);}
end        {OUT(END,NULL);}
if         {OUT(IF,NULL);}
{alpha}{alnum}* {OUT(ID,yytext);}
{digit}+   {OUT(INT,yytext);}
"<"       {OUT(LT,NULL);}
"<="      {OUT(LE,NULL);}
"="        {OUT(EQ,NULL);}
"<>"      {OUT(NE,NULL);}
">"       {OUT(GT,NULL);}
">="      {OUT(GE,NULL);}

```

本章总结（第二讲 p.18）

核心要点

1. 词法分析从字符流中寻找单词，输出**二元组序列**（种别码，属性值）
2. 高级语言的单词集合构成一个**3 型语言（正则语言）**
3. 3 型语言可以用 **RE**（正则表达式）、**RG**（正则文法）、**FA**（有穷自动机）描述
4. FA 的**状态转移图**直接指导词法分析器实现
5. 3 型语言理论催生了自动生成工具——**Lex / Flex**

知识结构图

